

POLITECNICO
MILANO 1863

# Astralog-HPC

## Requirement Analysis and Design

Alessandro Boschi, Ondřej Faiman, Aldas Lenkšas, Stanislaw Ostyk-Narbutt

June 11, 2026, v1.0

058170 - SOFTWARE ENGINEERING FOR HPC
High Performance Computing Engineering

# Contents

1. The project and project goals 2
1.1. Problem specification 2
2. Requirement analysis 3
2.1. Relevant human and non-human actors 3
2.2. Use cases 3
2.3. Domain assumptions 3
2.4. Requirements 4
2.4.1. Functional requirements 4
2.4.2. Non-functional requirements 4
3. Design 5
3.1. General description of the architecture 5
3.2. Sequence diagrams 6
3.3. Critical points and design decisions 7
4. Work distribution and effort 8
5. Usage of AI 9

## 1. The project and project goals

Modern space missions generate a massive, continuous stream of telemetry data. To ensure spacecraft safety, we focus on developing AstraLog-HPC, a software that identifies anomalies in aircraft subsystems. AstraLog-HPC acquires data streamed from sensors installed in each subsystem and executes a set of monitoring rules based on the absolute values of the received data, as well as their temporal variations and correlations. To simplify time management while keeping the problem realistic, we assume a unified synchronous system clock. Moreover, we assume that measurements across all sensors are equally spaced in time (e.g., exactly 1 measurement per second) and perfectly aligned (all sensors produce data for the entire mission duration without any interruptions). Therefore, the $n$-th row in each sensor stream corresponds to the same timestamp.

### 1.1. **Problem specification.**

Each spacecraft can contain multiple sensor types (e.g., temperature, pressure, voltage, ...), each potentially available in multiple instances. The specific type and number of sensors to be managed are available in a configuration file called `sensors.yaml` (to be made available by the instructors) and do not change during operation. Each sensor continuously produces data, and the spacecraft broadcasts a continuous data stream to the ground station. Each piece of data is transmitted as a JSON packet with the following fields: `timestamp`, `sensor_id`, `value`, and `priority`.

To align with real-world space communication, the tool must filter out packets that have malformed JSON, schema errors, or type errors.

Moreover, there are three rules that this project will handle: the simple "absolute threshold" rule that checks if a value has exceeded a prescribed threshold, a step difference rule examining if a change in immediately preceding sensor readings changes a certain value, a stateful rule violated when an anomaly occurs for consecutive measurements, and lastly a logical combinatorial rule that can group rules using `AND` and `OR` operands.

Each monitoring rule should belong to one of the four types above and should be identified by a unique identifier (`rule_id`). Optionally, a rule can have a priority, which can assume the values low, medium, or high. Such priority applies within a single rule type and means that rules with higher priority are evaluated before the others.

Logs, for both typical and violated rules, will be reported in separate output files.

The execution environments for AstraLog-HPC include classical machines/virtual machines, as well as an HPC cluster accessible through the SLURM job scheduler. As part of the project, we will define how the AstraLog-HPC components are deployed within these environments.

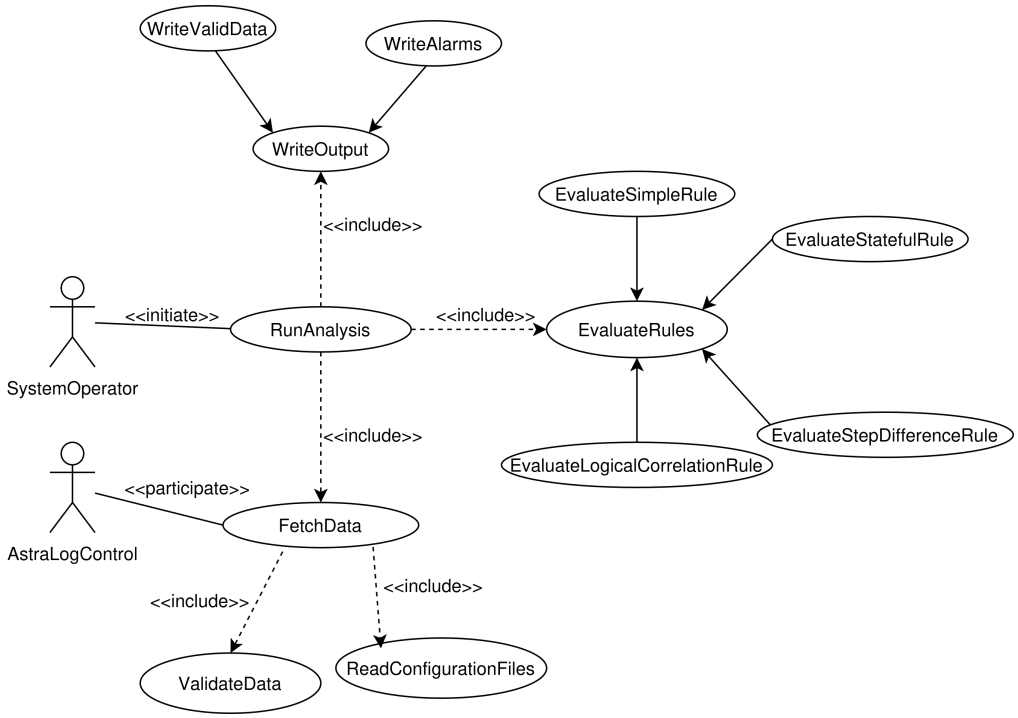
## 2. Requirement analysis

### 2.1. Relevant human and non-human actors.

- **System operator:** The operator that will launch the tool and monitor its outputs on the HPC cluster.
- **AstraLogControl (Spacecraft and its crew):** Requires strict monitoring of sensors for maintenance and control to ensure safety and integrity of the aircraft and its potential occupants or involved parties.

### 2.2. Use cases.

The primary use for the system is to monitor the spacecraft to ensure mission safety by having clear insight on any violations that could indicate failures or other problems in the spacecraft, where a system operator would monitor the output log files of the AstraLogControl spacecraft sensors.




### 2.3. Domain assumptions.

The domain assumptions are crucial given that space systems have special operating conditions and are advanced systems with complex internal sensors. Consequently, the following assumptions are outlined:

- Measurements across all sensors are equally spaced in time and perfectly aligned.
- The specific type and number of sensors do not change during operation, and they are provided in a configuration file called `sensors.yaml`.
- Each sensor is uniquely identified by a `sensor_id`.
- The rules have a fixed format. The set of rules is given in the `rules.json` file, and it can change during the mission.
- Each piece of data is transmitted as a JSON packet with a given structure.
- The incoming data may be corrupted.

### 2.4. **Requirements.**

The following requirements have been extracted from the problem description and discussions on various aspects of AstraLog-HPC. They have been split into functional and non-functional requirements.

#### 2.4.1. *Functional requirements.*

- The system must filter out syntactically invalid packets and save valid packets to a local batch file.
- The system must be able to read the data from a provided file.
- The system must be able to read incoming data from a live data stream.
- The system must correctly evaluate the four internal rules:

  Rule 1: Simple rule, absolute threshold passed.

  Rule 2: Step difference rule, immediately preceding measurements differ by a specified amount amount.

  Rule 3: Stateful rule, consecutive measurements are violated.

  Rule 4: Logical correlation rules, a combination via `AND`, `OR` operands across rules is violated.
- The system must evaluate rules in order of specified priority per rule: HIGH, MEDIUM, and LOW.
- The system must write the results of the evaluation of the rules in the correct format into the correct files.
- The system must allow the size of the local batch file containing the valid packets to be customized and specified.

#### 2.4.2. *Non-functional requirements.*

- The system must be runnable on classical machines/virtual machines.
- The system must support reading data via a HiveMQ MQTT Broker subscribed to a specific telemetry topic. machines, as well as on an HPC cluster accessible through the SLURM job scheduler.
- The system must filter out the incorrect data faster than the rate at which new data is coming in.

# 3. Design

## 3.1. General description of the architecture.

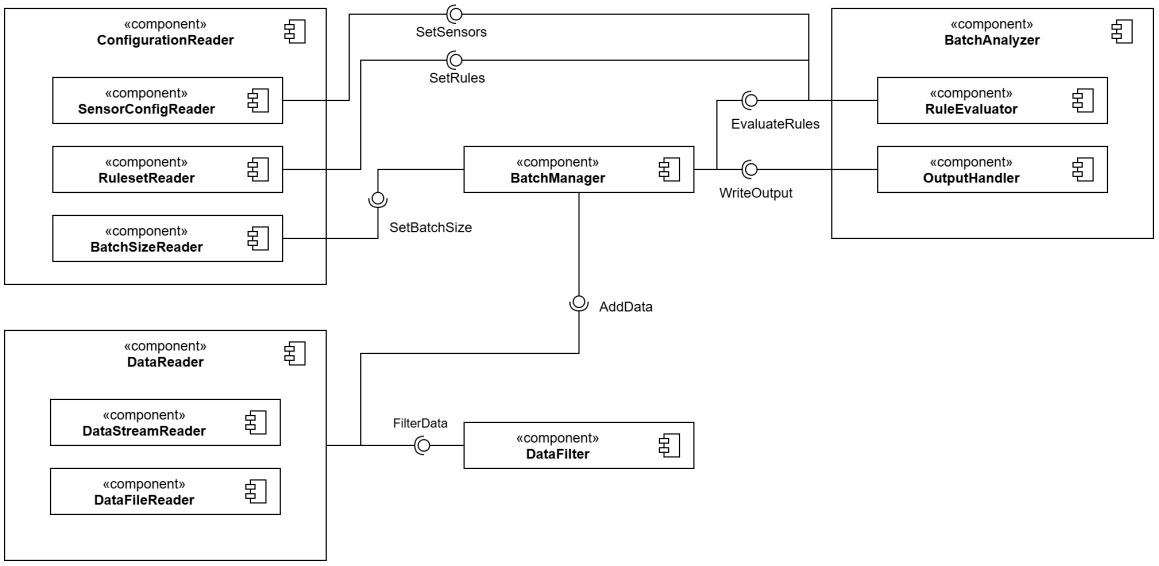



Figure 1. Component Diagram

The principal components of the proposed architecture depicted in Figure 1 are the `DataReader` and the `BatchAnalyzer`.
The purpose of the `DataReader` component is to acquire the sensor data coming either from a continuous stream coming from AstraLogControl or from file, for this reason those two functionalities are further decomposed in the sub-components `DataStreamReader` and `DataFileReader`.
Once a sensor reading has been obtained, it has to be filtered by the `DataFilter` component, whose purpose is to validate the received data, verifying that the payload is well-formed and with the correct JSON schema. If the data is valid, it is sent to the `BatchManager` component which will add an entry to the current batch until it reaches the desired size.
Once that happens, the `BatchAnalyzer` will be tasked with performing the analysis of the sensor data according to specifications, first evaluating the rules via the `RuleEvaluator` sub-component, then generating the correct output based on the result of the evaluation. This last operation is the responsibility of the `OutputHandler` sub-component.
Finally, the `ConfigurationReader` will manage the parsing of configuration files that specify the sensors present in the system, the rules to evaluate the correct functioning of the system and the desired batch size.

With this proposed architecture each component work is independent and abides to the single responsibility principle.

### 3.2. **Sequence diagrams.**

In this section we report two of the most important interactions between actors and components.

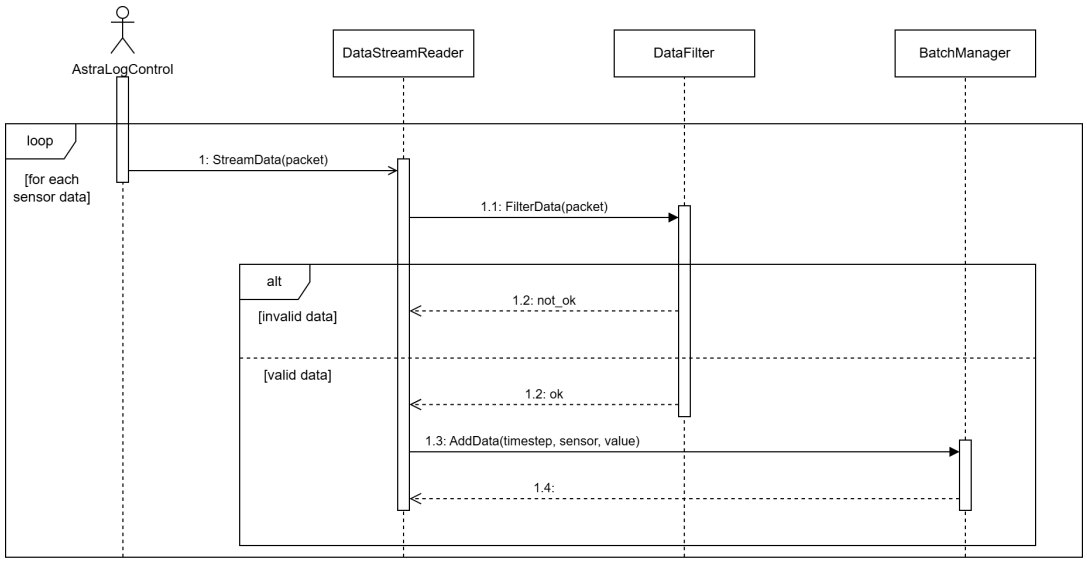



FIGURE 2. Component Diagram

First, in the interaction reported in Figure 2, the actor `AstraLogControl` continuously stream data to `AstraLog-HPC`. The packets are collected by the component `DataStreamReader`, which will delegate the validation of their content to the `DataFilter`. If the data is invalid no other action is performed by the system, which will then wait for more data coming from the sensors. Alternatively, if the received readings are valid, they have to be added to the current batch, so the values are sent to the `BatchManager`.

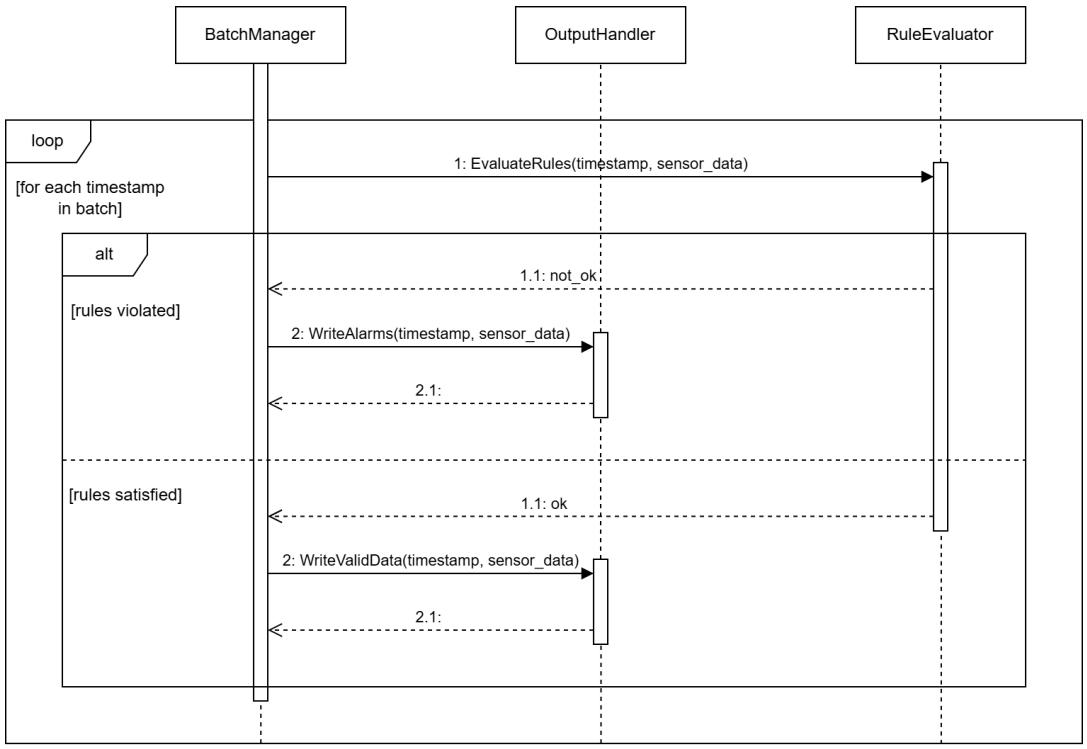



FIGURE 3. Component Diagram

The second interaction, depicted in Fig. 3, focuses on the analysis of a batch of readings. Once a batch is full, the `BatchManager` will iterate over all the timestamps present in the batch and for each of them the `RuleEvaluator` will evaluate all the rules provided via configuration files. If all the rules are satisfied at the current timestamp, the `OutputHandler` will be asked to report the sensors data to a csv file. Alternatively, the anomalous state of the system is reported in the system's log file.

### 3.3. **Critical points and design decisions.**

One of the most critical points in the design that needs particular attention is the parallelization of the computation. On one end, the requirements do not enforce the analysis of batches at a faster rate than they are produced, AstraLog-HPC just has to validate the received data and add it to the current batch before the next timestamp arrives. However, there is significant parallelism that can be exploited in the analysis of a batch.

**Pipeline**
The analysis is a pipeline composed of two consecutive steps: evaluation of the rules and writing to output files.
This enables us to partially overlap the analysis of two consecutive timestamps, i.e. while generating the output of a timestamp $t$ we can start evaluating the rules for timestamp $t + 1$.
From this consideration derives our decision of separating the `BatchAnalyzer` component in two distinct sub-components, each dedicated to one phase of the pipeline.

It should also be noted that when a violated rule is found, it's output can be performed immediately. On the other hand, when all rules are satisfied, we have to perform the output only after evaluating all of them.

**Rules**
Within a timestamp, simple, step difference and stateful rules can be evaluated fully in parallel, since they do not depend on other rules and they just read data from the batch.
Furthermore, logical correlation rules can also be performed in parallel to a certain extent. We can build a dependency graph where each node represents a rule of this type and the arcs indicate that the evaluation of the root node requires the evaluation of the leaf nodes. The graph construction stops when the node depends on rules of another type. This means that we can first evaluate all simple, step difference and stateful rules, then all the leaves of the graph can be evaluated parallel.

Another important consideration regarding stateful rules is that to efficiently compute them we have to keep track of the number of previous consecutive timestamps that violated the rule.

## 4. Work distribution and effort

| Name | Contribution | Time |
|---|---|---|
| Boschi Alessandro | Designing the program | 3h |
| | Writing design section | 5h |
| Ondřej Faiman | Designing the program | 2h |
| | Writing requirement analysis | 4h |
| Aldas Lenkšas | Designing the program | 2h |
| | Proof-reading and designing the document | 2h |
| Stanisław Ostyk-Narbutt | Designing the program | 2h |
| | Worked on requirements analysis and problem specification. | 4h |

## 5. Usage of AI

None.